

Phase 5 Structured Notes: Drawbacks of Two-Pointer Linear Queues

1. Current Progress Before Phase 5

Before Phase 5, you already learned the main queue ideas.

In **Phase 1**, you learned that a queue follows this rule:

FIFO = First In, First Out

This means the first value inserted into the queue is the first value removed from the queue.

You also learned that a normal queue has two ends:

Queue Part	Simple Meaning
front	The side where deletion happens
rear	The side where insertion happens

So in a normal queue:

Insert at rear.
Delete from front.

In **Phase 2**, you compared two array queue designs.

Design	Main Idea	Problem or Benefit
Single-pointer queue	Uses only rear	Deletion is slow because elements must shift
Two-pointer queue	Uses front and rear	Deletion is fast because only front moves

The most important Phase 2 idea was:

The two-pointer queue avoids shifting.

In **Phase 3**, you built a complete linear array queue in C using:

- struct Queue

- size
- front
- rear
- Q
- create()
- enqueue()
- dequeue()
- display()

You also learned these important conditions:

```
front == rear    // queue is empty
rear == size - 1 // queue is full in a linear array queue
```

In **Phase 4**, you wrote the same linear queue using C++ OOP.

You used:

- class Queue
- private data
- public methods
- constructors
- destructor
- new[]
- delete[]
- object syntax like `q.enqueue(10)`

The queue logic stayed the same. The organization became cleaner and safer.

Now Phase 5 asks an important question:

Even though a two-pointer linear queue is fast, what is wrong with it?

The answer is:

```
The problem is not speed.
The problem is memory reuse.
```

2. Main Goal of Phase 5

The goal of Phase 5 is to understand the weakness of a two-pointer linear array queue.

A two-pointer linear queue has fast operations:

```
enqueue() // O(1)
dequeue() // O(1)
```

But it has a serious memory problem.

The main problem is:

```
rear only moves forward.
```

Because `rear` only moves forward, the queue cannot reuse empty spaces at the beginning of the array after deletions.

This causes:

- memory wastage
- unused spaces at the front
- full-but-empty situations
- the need for a reset workaround
- the motivation for circular queues

3. What Is a Two-Pointer Linear Queue?

A two-pointer linear queue is an array-based queue that uses two index variables:

```
front
rear
```

The queue is called **linear** because `rear` moves in one direction only:

```
0 → 1 → 2 → 3 → 4
```

It does not naturally go back to the beginning.

It does **not** move like this:

```
0 → 1 → 2 → 3 → 4 → 0
```

That second movement is circular queue behavior, which comes in Phase 6.

4. Quick Reminder: Queue Structure

A linear array queue can be represented like this:

```
struct Queue {  
    int size;  
    int front;  
    int rear;  
    int *Q;  
};
```

Meaning of each member:

Member	Simple Meaning
size	Maximum number of values the queue can store
front	Index used for deletion
rear	Index used for insertion
Q	Pointer to the array that stores queue values

Important point:

front and rear are index values.
They are not actual memory addresses.

Example:

```
front = 0  
rear = 2
```

This means `front` and `rear` are positions inside the array.

5. Starting State of a Linear Queue

At the start, the queue is empty.

We write:

```
front = rear = -1;
```

This means:

```
front = -1  
rear  = -1
```

The queue is empty because:

```
front == rear
```

So the empty condition is:

```
front == rear
```

6. Full Condition in a Linear Queue

In a linear array queue, the queue is full when `rear` reaches the last valid index.

The last valid index is:

```
size - 1
```

So the full condition is:

```
rear == size - 1
```

Example:

```
size = 5  
last index = size - 1 = 4
```

So if:

```
rear = 4
```

then the queue is considered full.

7. Important Valid Range Rule

In this design, `front` does not directly point to the first valid element.

Instead:

```
front points before the first valid element.
```

So the first valid element is at:

```
front + 1
```

The last valid element is at:

```
rear
```

Therefore, the valid queue range is:

```
front + 1 to rear
```

Example:

```
front = 0  
rear  = 2  
Array = [10, 20, 30]
```

The valid range is:

```
front + 1 = 1  
rear = 2
```

So the logical queue is:

```
20 30
```

The value `10` may still exist physically in the array, but it is no longer part of the queue.

8. Logical Deletion

In a two-pointer linear queue, deletion is usually logical.

Logical deletion means:

The old value may still remain in the array, but the queue ignores it.

Example before deletion:

```
front = -1  
rear  = 2  
Array = [10, 20, 30]
```

The valid queue is:

10 20 30

Now we call:

```
dequeue();
```

The queue deletes `10` by moving `front` forward.

After deletion:

```
front = 0  
rear  = 2  
Array = [10, 20, 30]
```

The array may still show `10`, but the logical queue is:

20 30

Why?

Because the valid elements are from:

```
front + 1 = 1
```

to:

```
rear = 2
```

So the valid values are:

```
Q[1] = 20  
Q[2] = 30
```

The value `10` is ignored.

9. Why the Two-Pointer Queue Is Fast

The two-pointer queue is fast because deletion does not shift elements.

In a single-pointer queue, after deleting the first value, all remaining elements must move left.

But in a two-pointer queue, deletion only does this:

```
front++;
```

That is why:

```
dequeue() is O(1)
```

Also, insertion only moves `rear` and stores one value:

```
rear++;  
Q[rear] = x;
```

So:


```
enqueue() is 0(1)
```

This is good.

But Phase 5 teaches that speed is not the only issue.

10. Main Problem: Memory Wastage

The main weakness of the two-pointer linear queue is memory wastage.

Memory wastage happens because:

```
Deleted spaces at the beginning of the array are not reused.
```

Why are they not reused?

Because:

```
rear only moves forward.
```

It does not go back to earlier empty indexes.

Example array of size 5:

```
Index:  0   1   2   3   4
Array: [ ] [ ] [ ] [ ] [ ]
```

Insert 10, 20, 30:

```
Index:  0   1   2   3   4
Array: [10] [20] [30] [ ] [ ]

front = -1
rear  = 2
```

Now delete 10:

```
front = 0  
rear  = 2
```

Logical queue:

```
20 30
```

Index `0` is now empty logically.

But can `rear` use index `0` again?

```
No.
```

Because `rear` only moves forward.

So the next insertions go to index `3` and index `4`, not back to index `0`.

11. Single-Use Index Problem

In a simple linear queue, each array index can be used by `rear` only once.

That means:

```
Once rear passes an index, rear will not return to it.
```

Example:

```
rear moves like this:  
-1 → 0 → 1 → 2 → 3 → 4
```

After `rear` reaches `4`, it stops.

Even if indexes `0`, `1`, or `2` become empty later, `rear` does not go back to them.

This is why deleted spaces are wasted.

12. Simple Dry Run Showing Memory Wastage

Suppose the queue size is 5.

Starting state:

```
front = -1  
rear  = -1  
Array = [ ] [ ] [ ] [ ] [ ]
```

Insert 10:

```
front = -1  
rear  = 0  
Array = [10] [ ] [ ] [ ] [ ]
```

Insert 20:

```
front = -1  
rear  = 1  
Array = [10] [20] [ ] [ ] [ ]
```

Insert 30:

```
front = -1  
rear  = 2  
Array = [10] [20] [30] [ ] [ ]
```

Delete 10:

```
front = 0  
rear  = 2  
Array = [10] [20] [30] [ ] [ ]  
Logical queue = 20 30
```

Now insert 40:

```
front = 0  
rear  = 3
```

```
Array = [10] [20] [30] [40] [ ]  
Logical queue = 20 30 40
```

Now insert 50:

```
front = 0  
rear  = 4  
Array = [10] [20] [30] [40] [50]  
Logical queue = 20 30 40 50
```

Now the queue says it is full because:

```
rear == size - 1
```

That is:

```
rear = 4  
size - 1 = 4
```

But index `0` is not part of the logical queue anymore.

So there is wasted space at the beginning.

13. The Full-But-Empty Problem

The full-but-empty problem is the most important idea in Phase 5.

It happens when:

```
The queue has no valid elements, but insertion is still blocked.
```

This can happen because the queue may be empty according to `front == rear`, but full according to `rear == size - 1`.

The two conditions can both become true:

```
front == rear    // means empty  
rear == size - 1 // means full
```

This creates a contradiction.

The queue is logically empty, but it still looks full.

14. Full-But-Empty Dry Run

Suppose the queue size is 5.

Starting state:

```
front = -1
rear  = -1
Array = [ ] [ ] [ ] [ ] [ ]
```

Insert 10, 20, 30, 40, 50:

```
Index:  0    1    2    3    4
Array: [10] [20] [30] [40] [50]

front = -1
rear  = 4
```

Now the queue is full because:

```
rear == size - 1
```

Now delete all values one by one.

Delete 10:

```
front = 0
rear  = 4
Logical queue = 20 30 40 50
```

Delete 20:

```
front = 1
rear  = 4
Logical queue = 30 40 50
```

Delete 30:

```
front = 2
rear  = 4
Logical queue = 40 50
```

Delete 40:

```
front = 3
rear  = 4
Logical queue = 50
```

Delete 50:

```
front = 4
rear  = 4
Logical queue = empty
```

Now the queue is empty because:

```
front == rear
```

But the queue also looks full because:

```
rear == size - 1
```

So we have this situation:

```
front == rear    → empty
rear == size - 1 → full
```

The array has no valid queue elements, but the next insertion is blocked because `rear` is already at the last index.

This is called the **full-but-empty scenario**.

15. Why Full-But-Empty Is a Problem

The problem is that the queue cannot accept new values even though all old values are gone.

The array may look like this:

```
Index:  0    1    2    3    4
Array: [10] [20] [30] [40] [50]
```

But logically, the queue is empty.

Why?

Because:

```
front = 4
rear  = 4
```

There are no valid elements between `front + 1` and `rear`.

However, if we try to insert a new value, the queue checks:

```
rear == size - 1
```

Since:

```
rear = 4
size - 1 = 4
```

the queue says:

```
Queue is Full
```

That is incorrect from a practical point of view because the queue is empty.

The problem is caused by the linear movement of `rear`.

16. Beginner Analogy

Imagine a room with 5 chairs.

```
Chair:  0   1   2   3   4
```

Five people sit down.

Then all five people leave.

Now all chairs are empty.

But the manager says:

“Sorry, no one else can sit. I already reached chair 4.”

That sounds wrong because the chairs are empty.

But this is exactly what a linear queue does.

It refuses to reuse the earlier positions because `rear` does not go back.

17. Why Resetting Pointers Is Introduced

To reduce the full-but-empty problem, we can use a simple workaround:

```
When the queue becomes completely empty, reset front and rear to -1.
```

This makes the queue look brand new again.

Before reset:

```
front = 4
rear  = 4
```

After reset:

```
front = -1
rear  = -1
```


Now the next insertion can start at index `0` again.

18. Reset Rule

The reset rule is:

If a deletion makes front equal to rear, reset both to -1.

In code:

```
if (q->front == q->rear) {
    q->front = q->rear = -1;
}
```

Simple meaning:

If the queue becomes empty, restart it.

19. Dequeue With Reset Code

```
int dequeueWithReset(struct Queue *q) {
    int x = -1;

    if (q->front == q->rear) {
        printf("Queue is Empty\n");
    } else {
        q->front++;
        x = q->Q[q->front];

        if (q->front == q->rear) {
            q->front = q->rear = -1;
        }
    }

    return x;
}
```

20. Line-by-Line Explanation of `dequeueWithReset()`

Step 1: Create a default return value

```
int x = -1;
```

This means if deletion fails, the function returns `-1`.

Step 2: Check whether the queue is empty

```
if (q->front == q->rear) {  
    printf("Queue is Empty\n");  
}
```

If `front` and `rear` are equal, there is no valid element to delete.

Step 3: Move `front` forward

```
q->front++;
```

This moves `front` to the value that should be deleted.

Step 4: Store the deleted value

```
x = q->Q[q->front];
```

This saves the value that is being removed from the queue.

Step 5: Check whether the queue became empty

```
if (q->front == q->rear) {  
    q->front = q->rear = -1;  
}
```

After deletion, if `front` and `rear` are equal, the queue has no more valid elements.

So we reset both pointers to `-1`.

Step 6: Return the deleted value

```
return x;
```

This returns the removed value.

21. Reset Dry Run

Suppose the queue size is 3.

Start:

```
front = -1  
rear  = -1  
Array = [ ] [ ] [ ]
```

Insert 10:

```
front = -1  
rear  = 0  
Array = [10] [ ] [ ]
```

Insert 20:

```
front = -1  
rear  = 1  
Array = [10] [20] [ ]
```

Insert 30:

```
front = -1  
rear  = 2  
Array = [10] [20] [30]
```

Now delete 10:

```
front = 0
rear  = 2
Logical queue = 20 30
```

The queue is not empty yet, so no reset happens.

Delete 20:

```
front = 1
rear  = 2
Logical queue = 30
```

The queue is still not empty, so no reset happens.

Delete 30:

```
front = 2
rear  = 2
Logical queue = empty
```

Now:

```
front == rear
```

So reset happens:

```
front = -1
rear  = -1
```

Now insert 40:

```
rear moves from -1 to 0
Q[0] = 40
```

New state:

```
front = -1
rear  = 0
```

```
Array = [40] [20] [30]  
Logical queue = 40
```

The old physical values `20` and `30` may still be in memory, but they are ignored.

The valid queue is only:

```
40
```

22. Why Reset Helps

Reset helps when the queue becomes completely empty.

Without reset:

```
front = 2  
rear  = 2
```

The queue is empty, but `rear` is at the end.

With reset:

```
front = -1  
rear  = -1
```

Now the queue can start inserting again from index `0`.

So reset helps solve the full-but-empty case.

23. Important Limitation of Reset

Reset does not solve all memory wastage problems.

Reset only works when the queue becomes completely empty.

It does not help when the queue is only partially empty.

This is very important.

24. Partial Empty Problem

Suppose the queue size is 5.

Insert 10, 20, 30, 40, 50:

```
Index:  0    1    2    3    4
Array: [10] [20] [30] [40] [50]

front = -1
rear  = 4
```

Now delete four values: 10, 20, 30, and 40.

After deleting 10:

```
front = 0
rear  = 4
Logical queue = 20 30 40 50
```

After deleting 20:

```
front = 1
rear  = 4
Logical queue = 30 40 50
```

After deleting 30:

```
front = 2
rear  = 4
Logical queue = 40 50
```

After deleting 40:

```
front = 3
rear  = 4
Logical queue = 50
```

Now only **50** remains.

Indexes **0**, **1**, **2**, and **3** are no longer part of the queue.

They are wasted spaces.

But can we reset?

No.

Why?

Because the queue is not completely empty.

```
front = 3  
rear  = 4
```

So:

```
front != rear
```

There is still one valid element: 50 .

Can we insert a new value?

No.

Why?

Because:

```
rear == size - 1
```

So the queue says it is full.

This shows the limitation of resetting pointers.

25. Why Reset Is Not a Complete Solution

Reset only works when:

```
front == rear
```

That means the queue is fully empty.

But if even one element remains, reset cannot happen.

Example:

```
front = 3  
rear  = 4  
Logical queue = 50
```

The queue has many wasted spaces at the beginning:

Indexes 0, 1, 2, and 3 are wasted.

But the queue cannot reset because `50` is still valid.

So reset is only a small workaround.

It is not the real solution.

26. Real Solution: Circular Queue

The real solution is a circular queue.

A circular queue allows `rear` to wrap around to the beginning of the array.

Instead of stopping at the last index, `rear` can move like this:

```
0 → 1 → 2 → 3 → 4 → 0 → 1 → 2 ...
```

This lets the queue reuse empty spaces at the beginning.

That is why Phase 6 comes after Phase 5.

Phase 5 shows the problem.

Phase 6 shows the solution.

27. Complete Beginner C Program for Phase 5

```
#include <stdio.h>
#include <stdlib.h>

struct Queue {
    int size;
    int front;
    int rear;
    int *Q;
};

void create(struct Queue *q, int size) {
    q->size = size;
    q->front = q->rear = -1;
    q->Q = (int *)malloc(size * sizeof(int));
}

void enqueue(struct Queue *q, int x) {
    if (q->rear == q->size - 1) {
        printf("Queue is Full\n");
    } else {
        q->rear++;
        q->Q[q->rear] = x;
    }
}

int dequeueWithReset(struct Queue *q) {
    int x = -1;

    if (q->front == q->rear) {
        printf("Queue is Empty\n");
    } else {
        q->front++;
        x = q->Q[q->front];

        if (q->front == q->rear) {
            q->front = q->rear = -1;
        }
    }

    return x;
}

void display(struct Queue q) {
    if (q.front == q.rear) {
```

```

        printf("Queue is Empty\n");
        return;
    }

    for (int i = q.front + 1; i <= q.rear; i++) {
        printf("%d ", q.Q[i]);
    }
    printf("\n");
}

int main() {
    struct Queue q;
    create(&q, 3);

    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);

    display(q);

    printf("Deleted: %d\n", dequeueWithReset(&q));
    printf("Deleted: %d\n", dequeueWithReset(&q));
    printf("Deleted: %d\n", dequeueWithReset(&q));

    enqueue(&q, 40);
    display(q);

    free(q.Q);
    return 0;
}

```

28. Program Explanation

Header files

```

#include <stdio.h>
#include <stdlib.h>

```

`stdio.h` is used for `printf()`.

`stdlib.h` is used for `malloc()` and `free()`.

Queue structure

```
struct Queue {  
    int size;  
    int front;  
    int rear;  
    int *Q;  
};
```

This stores all queue information in one structure.

Create function

```
void create(struct Queue *q, int size) {  
    q->size = size;  
    q->front = q->rear = -1;  
    q->Q = (int *)malloc(size * sizeof(int));  
}
```

This function:

1. stores the queue size
 2. sets `front` and `rear` to `-1`
 3. creates the array in heap memory
-

Enqueue function

```
void enqueue(struct Queue *q, int x) {  
    if (q->rear == q->size - 1) {  
        printf("Queue is Full\n");  
    } else {  
        q->rear++;  
        q->Q[q->rear] = x;  
    }  
}
```

This function inserts a value at the rear.

It first checks whether the queue is full.

If not full:

```
q->rear++;  
q->Q[q->rear] = x;
```

Dequeue with reset function

```
int dequeueWithReset(struct Queue *q)
```

This function deletes a value from the front.

If the queue becomes empty after deletion, it resets:

```
q->front = q->rear = -1;
```

Display function

```
void display(struct Queue q)
```

This function prints the logical queue elements.

It starts from:

```
q.front + 1
```

and stops at:

```
q.rear
```

Main function

```
int main() {  
    struct Queue q;  
    create(&q, 3);  
}
```

This creates a queue of size 3.

Then:

```
enqueue(&q, 10);  
enqueue(&q, 20);  
enqueue(&q, 30);
```

The queue becomes:

```
10 20 30
```

Then:

```
dequeueWithReset(&q);  
dequeueWithReset(&q);  
dequeueWithReset(&q);
```

This deletes all values.

After the last deletion, reset happens.

Then:

```
enqueue(&q, 40);
```

This inserts `40` at index `0` because the queue was reset.

Finally:

```
free(q.Q);
```

This releases heap memory.

29. Expected Output

```
10 20 30  
Deleted: 10  
Deleted: 20
```

```
Deleted: 30  
40
```

This output proves that reset allows the queue to start again after it becomes completely empty.

30. Full Program Dry Run

Step 1: Create queue

```
create(&q, 3);
```

State:

```
size = 3  
front = -1  
rear = -1  
Array = [ ] [ ] [ ]
```

Step 2: Insert 10

```
enqueue(&q, 10);
```

State:

```
front = -1  
rear = 0  
Array = [10] [ ] [ ]  
Logical queue = 10
```

Step 3: Insert 20

```
enqueue(&q, 20);
```

State:

```
front = -1
rear = 1
Array = [10] [20] [ ]
Logical queue = 10 20
```

Step 4: Insert 30

```
enqueue(&q, 30);
```

State:

```
front = -1
rear = 2
Array = [10] [20] [30]
Logical queue = 10 20 30
```

Now the queue is full because:

```
rear = 2
size - 1 = 2
```

Step 5: Delete 10

```
dequeueWithReset(&q);
```

State:

```
front = 0
rear = 2
Logical queue = 20 30
```

No reset happens because the queue is not empty.

Step 6: Delete 20

```
dequeueWithReset(&q);
```

State:

```
front = 1  
rear = 2  
Logical queue = 30
```

No reset happens because the queue is not empty.

Step 7: Delete 30

```
dequeueWithReset(&q);
```

State before reset:

```
front = 2  
rear = 2  
Logical queue = empty
```

Now:

```
front == rear
```

So reset happens.

State after reset:

```
front = -1  
rear = -1  
Logical queue = empty
```

Step 8: Insert 40

```
enqueue(&q, 40);
```

State:

```
front = -1  
rear = 0  
Array = [40] [20] [30]  
Logical queue = 40
```

The old values `20` and `30` may still physically exist, but they are ignored.

31. Complexity Summary

Operation	Time Complexity	Why?
<code>enqueue()</code>	$O(1)$	Moves <code>rear</code> and stores one value
<code>dequeueWithReset()</code>	$O(1)$	Moves <code>front</code> , returns one value, and maybe resets pointers
<code>display()</code>	$O(n)$	Prints all valid elements
Reset check	$O(1)$	Only compares <code>front</code> and <code>rear</code>

Space complexity:

```
O(size)
```

The queue uses an array of the chosen size.

32. Key Phase 5 Concepts

Concept	Simple Meaning
Linear queue	A queue where <code>rear</code> moves forward in a straight line
Memory wastage	Empty spaces at the beginning cannot be reused
Logical deletion	Deletion happens by moving <code>front</code> , not by erasing the value physically

Concept	Simple Meaning
Full condition	<code>rear == size - 1</code>
Empty condition	<code>front == rear</code>
Full-but-empty problem	The queue is empty but still appears full because <code>rear</code> is at the last index
Resetting pointers	Setting <code>front</code> and <code>rear</code> back to <code>-1</code> when the queue becomes empty
Reset limitation	Reset only works when the queue is completely empty
Circular queue motivation	Circular queue solves memory wastage by wrapping around

33. Common Beginner Mistakes

Mistake 1: Thinking the two-pointer queue has no problem

The two-pointer queue is faster than the single-pointer queue, but it still wastes memory.

Correct idea:

Two-pointer queue fixes shifting, not memory reuse.

Mistake 2: Thinking deleted values are physically removed

After dequeue, the old value may still be in the array.

Correct idea:

The value is removed logically because front moved past it.

Mistake 3: Thinking empty front spaces can automatically be reused

In a linear queue, `rear` does not go back to the beginning.

Correct idea:

Deleted front spaces are wasted unless the queue is reset or made circular.

Mistake 4: Thinking reset solves everything

Reset only works when the queue is fully empty.

Correct idea:

Reset does not help if even one element remains in the queue.

Mistake 5: Resetting too early

Do not reset while there are still valid elements.

Wrong:

Resetting when `front != rear`

Correct:

```
if (front == rear) {  
    front = rear = -1;  
}
```

Mistake 6: Forgetting the full condition

The full condition in a linear queue is:

```
rear == size - 1
```

This is the reason the queue may say full even when earlier spaces are empty.

Mistake 7: Forgetting why circular queue is needed

Circular queues are introduced because linear queues waste space.

Correct idea:

Phase 5 shows the problem.
Phase 6 shows the solution.

34. Phase 5 Mini Review Questions

Question 1

What is the main weakness of a two-pointer linear queue?

Answer:

It cannot reuse deleted spaces at the beginning of the array.

Question 2

Why does memory wastage happen?

Answer:

Because `rear` only moves forward and does not go back to earlier empty indexes.

Question 3

What is the empty condition?

Answer:

`front == rear`

Question 4

What is the full condition in a linear array queue?

Answer:

```
rear == size - 1
```

Question 5

What is the full-but-empty problem?

Answer:

It is when the queue is logically empty but still appears full because `rear` is at the last index.

Question 6

When should we reset `front` and `rear`?

Answer:

When a deletion makes the queue completely empty.

```
if (front == rear) {  
    front = rear = -1;  
}
```

Question 7

Does reset solve all memory wastage problems?

Answer:

No. Reset only works when the queue becomes completely empty.

Question 8

Why does reset not help when one element remains near the end?

Answer:

Because the queue is not empty, so resetting would lose the remaining valid element.

Question 9

What data structure solves this problem better?

Answer:

A circular queue.

Question 10

Why does a circular queue solve the memory wastage problem?

Answer:

Because `rear` can wrap around to the beginning and reuse empty spaces.

35. Phase 5 Final Summary

In Phase 5, you learned the drawbacks of a two-pointer linear queue.

A two-pointer queue is fast because:

```
enqueue() is O(1)
dequeue() is O(1)
```

It avoids shifting by moving `front` forward during deletion.

But it has a weakness:

```
rear only moves forward.
```

Because of this, deleted spaces at the beginning of the array are wasted.

The major problem is the full-but-empty scenario:

```
front == rear    // queue is empty
rear == size - 1 // queue appears full
```

This means the queue may have no valid elements, but it still refuses new insertions.

A simple workaround is resetting pointers:

```
if (front == rear) {  
    front = rear = -1;  
}
```

But reset only helps when the queue becomes completely empty.

If one element still remains near the end, reset cannot happen, and earlier spaces are still wasted.

So the final lesson is:

Linear queues are fast, but they do not reuse deleted spaces well.

That is why we need the next topic:

Circular Queue

A circular queue fixes this problem by allowing `rear` to wrap around and reuse empty spaces at the beginning of the array.

36. Final Takeaway

The most important Phase 5 idea is:

A two-pointer linear queue solves the shifting problem, but it does not solve the memory wastage problem.

Remember:

```
front == rear      means empty  
rear == size - 1   means full  
rear only moves forward in a linear queue  
reset helps only when the queue becomes fully empty  
circular queue is needed to reuse wasted spaces properly
```

Simple final statement:

A linear queue works, but it wastes space. A circular queue is introduced because it can reuse that wasted space.